

Solartis AI Engineering Internship – 3 Day Challenge

About Solartis – www.solartis.com

Solartis is building an AI-native Policy Administration Platform (PAS) for insurance carriers, MGAs, and wholesalers.

Our platform is evolving toward:

- Model Context Protocol (MCP)-driven systems
- AI-assisted product configuration
- RAG (Retrieval-Augmented Generation) based intelligence
- AI-driven monitoring and anomaly detection

This assignment reflects real-world engineering problems we solve.

1 Duration

- Total Time: **3 Days (72 hours)**
- Recommended Effort: **8–12 hours per day**

2 Objective & Expectations

Build a **Mini AI System** that can:

1. Understand natural language queries
2. Retrieve relevant context using **RAG (Vector Database)**
3. Analyze system/query performance issues
4. Suggest improvements or actions
5. (Optional) Call a mock API (MCP-style interaction)

2.1 Who Should Take This Challenge

This challenge is intended for candidates who:

- Enjoy solving open-ended problems
- Are curious about AI systems (RAG, LLMs, etc.)
- Are comfortable learning independently
- Are willing to experiment and figure things out

2.2 This Challenge May NOT Be Suitable If

- You prefer step-by-step instructions
- You are looking only for structured training
- You are not comfortable with ambiguity
- You are not interested in AI or system design

We strongly encourage only motivated candidates to attempt this.

2.3 Minimum Expectation

At a minimum, your submission must:

- Work end-to-end (query → retrieval → response)
- Use at least 3 dataset cases
- Provide a clear explanation for each response

Submissions without a working flow will not be evaluated.

2.4 What Differentiates Strong Candidates

Strong submissions will demonstrate:

- Clear reasoning and structured thinking
- Ability to connect multiple concepts (RAG + system design)
- Thoughtful improvements beyond the problem statement
- Curiosity and experimentation

We value thinking more than completeness.

2.5 Open-Ended Question (Mandatory)

Answer the following in your README:

“If you were designing this system for production at scale, what would you change or improve?”

2.6 Important Note

We are more interested in:

- Your effort
- Your thought process
- Your willingness to learn

than in perfect solutions.

2.7 Dataset Access

You will be provided with a sample dataset (JSON / text format) containing:

- Query examples
- Execution metrics
- Context information

You may structure and store this dataset as needed for your solution.

3 Dataset

You will be provided with a small dataset (JSON/text format) containing performance scenarios.

3.1 Sample Schema (For Understanding Only)

Table: policy_data

- policy_id (int)
- state (string)
- premium_amount (decimal)
- status (string)
- created_date (date)

Table: claims_data

- claim_id
- policy_id
- claim_amount

3.2 Sample JSON Structure

```
{
  "policy": {
    "state": "CA",
    "premium": 1200,
    "status": "ACTIVE"
  }
}
```

3.3 Example Cases

3.3.1 Case 1: Full Table Scan

Query:

```
SELECT * FROM policy_data
```

Execution Time: very high

Context:

- Large dataset
- No filtering

3.3.2 Case 2: JSON Filtering

Query:

```
SELECT * FROM policy_data  
WHERE JSON_EXTRACT(data, '$.policy.state') = 'CA'
```

Execution Time: 25 sec

Context:

- No index on JSON field

3.3.3 Case 3: High Frequency Query

Query:

```
SELECT * FROM config_table WHERE key = ?
```

Execution Time: low

Frequency: very high

3.3.4 Case 4: Complex Join

Query:

JOIN multiple tables with JSON extraction

Execution Time: medium

3.3.5 Case 5: Anomaly Scenario

Latency spike from 1 sec → 50 sec

You may use at least 3–5 cases for your solution.

You are given a dataset derived from real-world system behavior.

Example: `SELECT policy_id, premium_amount, state FROM policy_data WHERE status = 'ACTIVE'`

Example Schema Table: `policy_data`

- `policy_id` (int)
- `state` (string)
- `premium_amount` (decimal)
- `status` (string)
- `created_date` (date)

Table: `claims_data`

- `claim_id`
- `policy_id`
- `claim_amount`

JSON Structure Sample

```
{
  "policy": {
    "state": "CA",
    "premium": 1200,
    "status": "ACTIVE"
  }
}
```

Case: JSON Filtering

Query:

`SELECT * FROM policy_data WHERE JSON_EXTRACT(data, '$.policy.state') = 'CA'`

Execution Time: 25 sec

Context:

- No index on JSON field
- Large dataset

Each case includes:

- Query pattern
- Execution time
- Frequency
- Context

Mock APIs:

GET `/analyze/query`
POST `/detect/anomaly`

GET /suggest/optimization

3.4 Example Case

Query:

```
SELECT * FROM large_transaction_table
```

Execution Time: 2469 sec

Context:

- Large table
- No filtering
- Full table scan

3.5 Example Queries You Should Handle

Your system should be able to answer:

- “Why is this query slow?”
- “How can this query be optimized?”
- “Is this an anomaly?”
- “What would you change in system design?”

4 Dataset Cases (Use at least 3–5)

4.1 Case 1: Full Table Scan

```
SELECT * FROM large_table
```

Latency: very high

4.2 Case 2: JSON Field Filtering

Query uses JSON_EXTRACT in WHERE clause

4.3 Case 3: Complex Join + JSON Processing

Multiple joins + JSON_TABLE

4.4 Case 4: High Frequency Query

Fast query but executed thousands of times

4.5 Case 5: Nested Subquery

Subquery inside WHERE

4.6 Case 6: Aggregation Query

COUNT with multiple joins

4.7 Case 7: Logging Table Query

Querying large logging table

4.8 Case 8: Knowledge Base Query (Configuration Driven System)

4.9 Case 9: Update Query Performance

4.10 Case 10: Anomaly Scenario

Latency spike from normal → abnormal

5 What You Need to Build

Your system should include:

5.1 RAG Layer

- Store dataset in a vector database
- Retrieve relevant cases based on user query

5.2 AI Processing Layer

- Interpret user query
- Use retrieved context
- Generate explanation

5.3 Output

Return structured response:

```
{
  "problem": "Full table scan",
  "root_cause": "No filtering/index",
  "suggestion": "Add WHERE clause and index",
  "confidence": "high"
}
```

```
}
```

5.4 Mock API (Optional)

You may simulate API calls such as:

- /analyze/query
- /detect/anomaly
- /suggest/optimization

Note: You do NOT need to build real APIs. Simple function calls or mock responses are sufficient.

6 Use of AI Tools

You are allowed to use:

- ChatGPT
- Claude
- GitHub Copilot
- Any AI coding assistant

6.1 Mandatory Requirement

Include in README:

6.1.1 AI Usage Disclosure

- Tools used
- What you used them for
- What you changed/improved
- What challenges you faced

6.2 Disqualification

- Copy-paste without understanding
- Cannot explain your solution
- No working system

7 Deliverables

7.1 Source Code

- GitHub repository (preferred)

7.2 README (VERY IMPORTANT)

Include:

- Problem understanding
- Architecture diagram (simple is fine)
- How to run
- Design decisions
- Trade-offs
- Improvements you would make

7.3 Demo (Optional)

- 5–10 min video explaining your system

8 Evaluation Criteria

Criteria	Weight
Problem Solving	40%
Working System	30%
Thinking & Curiosity	20%
Communication	10%

9 What We Value

- Clear thinking
- Working solutions
- Curiosity
- Ownership

10 What We Don't Value

- Over-engineering
- Copy-paste solutions
- Fancy UI without logic

11 Tips

- Start simple → then improve
- Focus on end-to-end flow
- Show your thinking clearly

12 Bonus (Optional)

- Detect anomalies automatically
- Suggest system-level improvements
- Use smaller/open models (SLM)

13 Submission Deadline

Please submit your solution within **3 days from the time you receive this assignment**.

13.1 Questions / Clarifications

If you have questions, please reach out via:

Sunil N <sunil_n@solartis.com>

Kishore Balakrishnan <kishore_b@solartis.com>

14 Final Note

This is not an exam.

This is an opportunity to demonstrate:

- How you think
- How you build
- How you learn

We are looking for engineers who can build **AI-first systems**.

Good luck!

Solartis Engineering Team